YojanaBench — Project Explained in Simple Language

A plain-English guide for every team member. No prior machine-learning knowledge needed. Read this end to end and you will understand what the project does, how it is built, and how the pieces talk to each other.

---

1. What is this project? (the one-paragraph version)

YojanaBench is a software project about **Indian government welfare schemes** (like old-age pension, farmer income support, housing, scholarships, etc.).

It does **two big jobs**:

1 **Audit job** — You give it a person's details and documents, and it tells you *whether they are eligible* for a scheme, *how much money they should get*, and *which documents are missing*. It shows the exact rule ("clause") it used, so you can trust the answer. It can also score itself against a test dataset.

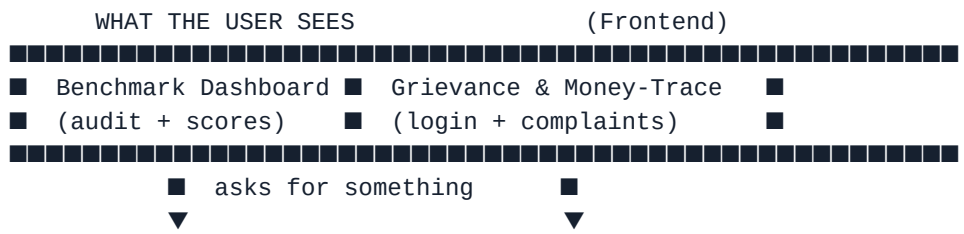
1 **Money-trace job** — A citizen says *"the government says it paid me, but I never got the money."* The system **traces the payment** and tells you what happened: was it really credited to their account, was it only partly paid, or was it **diverted/stolen** to someone else's account? It even detects a **fraud ring** (one account collecting money for many people).

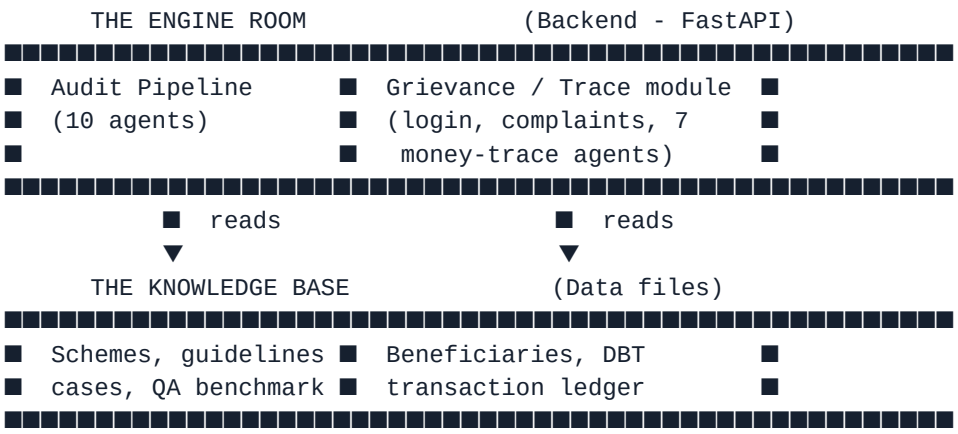
Everything runs **on your own laptop, offline, with zero cost**. No paid AI, no API keys, no internet needed after setup.

---

2. The big picture

Think of the project as a factory with four floors:





The **frontend** is what you click. The **backend** does the thinking. The **data files** are the knowledge the backend reads.

---

3. Part A — The Dataset (the knowledge base)

This is the "study material" the system learns from and reasons over. It lives in the `data/` folder.

- `data/schemes/` — 10 JSON files, one per scheme. Each file lists the rules: who is eligible (conditions), how much benefit they get (formula), and which documents are required. Example: old-age pension needs age ≥ 60 and BPL status.

- `data/guidelines/` — 10 text documents that look like official rulebooks. They are split into small numbered pieces called **clauses** (e.g. `IGNOAPS-E1`). The system quotes these clauses as evidence for every decision.

- `data/cases/` — 28 ready-made example applications (people) used for the live demo, so you can click an audit without typing anything.

- `data/qa/qa_benchmark.json` — 100 test questions and their correct answers. This is the "exam paper" used to score the system. Categories: eligibility, entitlement, documents, appeal.

- `data/beneficiaries.json` — 12 fake citizens (B-0001 to B-0012) with their bank account details. Used only by the money-trace module.

- `data/dbt/transactions.json` — the **payment ledger**: a list of payments the government claims it made, and where the money actually went. This is the "ground truth" the trace agents uncover. It secretly includes stolen-money and

fraud-ring cases.

- `data/meta/leaderboard.json` — standings table showing how well different models scored on the benchmark.

---

4. Part B — The Audit Pipeline (10 agents)

What is an "agent"? In this project an *agent* is simply a small, focused function (a step) that does one job and passes its result to the next step. It is **not** a scary AI robot — most of them are plain Python code. Chaining the steps is called a **pipeline**.

The audit pipeline has **10 agents**, run in this order:

1 **Corpus agent** — Loads all schemes and guideline documents, chops them into clauses, and builds a search index.

1 **Intake agent** — Takes the raw application, cleans it up, figures out which scheme it belongs to, and lists which documents are missing.

1 **RAG agent** — Search specialist. Finds the most relevant rule clauses for this application (RAG = "Retrieval-Augmented Generation"; here it just means *find the right rules first*).

1 **Entitlement agent** — The calculator. Checks each eligibility condition and computes the benefit amount **in code**, so numbers can never be made up.

1 **Verification agent** — The double-checker. Re-does the eligibility decision independently. If the two disagree, it raises a flag.

1 **Orchestrator agent** — The boss. Combines everything into one final decision (a **verdict**) plus a confidence score, and decides if a human must review.

1 **Evaluation agent** — The examiner. Compares the system's answers with the correct answers and computes scores (accuracy, error, etc.).

1 **Appeal agent** — If rejected, it decides which appeal channel to use and drafts an appeal letter.

1 **Counterfactual advisor** — The coach. Says *"if you change X and Y, you become eligible"*, but never suggests changing things that cannot change (like age).

1 **Document-gap analyzer** — The checklist. Ranks the missing documents and says honestly whether submitting them will actually change the verdict.

Key idea: the important decisions (eligible? how much?) are done by **plain rules in code**, not by an AI that might hallucinate. The AI (optional) only makes the wording nicer.

The verdict (final decision)

Every audit ends in one of three outcomes:

- **eligible** — qualifies,
- **ineligible** — does not qualify,
- **needs_info** — not enough information; ask for more.

---

5. Part C — The Grievance & Money-Trace module

The problem it solves

Welfare money is paid by **DBT = Direct Benefit Transfer** (money sent straight to the citizen's bank account). Sometimes the citizen never receives it. Before this module, nobody could tell *where the money went*. Now the system traces it.

Logins (two types of users)

- **Beneficiary (citizen)** — can file a complaint and see only their own cases.
- **Official (government officer)** — can see and act on every complaint.

Passwords are stored safely (scrambled with a technique called **PBKDF2**), and after login the user gets a **token** (like a temporary ID card) so they don't type the password again and again.

Demo accounts to try:

- Citizen whose money was **stolen**: `geeta / geeta123`
- Citizen whose money **arrived fine**: `ravi / ravi123`
- Officer: `officer / officer123`

The 7 money-trace agents

When a complaint is filed, these run in order to trace the money:

1. **Intake** — Understands the complaint and finds the person in the registry.
2. **DBT lookup** — Finds the government's payment record (using the **UTR**, a unique payment reference number, or the person + scheme).

1. **Credit check** — The heart of the module. Did the money land in the citizen's **own** account and under their **own** name? Or somewhere else?
1. **Reconcile** — Compares the amount the government *sanctioned* (approved) with the amount *actually credited*. Finds any shortfall.
1. **Fraud detector** — Looks for foul play: money sent to a wrong account, the same account collecting for many different people (**mule account / fraud ring**), duplicate reference numbers, or payments made to companies instead of people.
1. **Orchestrator** — Combines the findings into one clear result with a confidence score and a recommended action.
1. **Grievance notice** — Automatically drafts the escalation letter to send to the bank or scheme authority, with a deadline.

The possible money-trace verdicts

- **received** — money reached the citizen's own account. All good.
- **not_credited** — government marked it paid, but no money arrived.
- **partial** — only part of the money arrived.
- **diverted** — money went to the wrong account (**suspected theft**).
- **failed** — the transfer failed and bounced back to the government.
- **multiple** — more than one payment matches; need to confirm which one.
- **not_found** — no government payment record exists for this claim.
- **needs_info** — not enough details to trace.

---

6. Part D — The Backend (the engine room)

The backend is written in **Python** using a web framework called **FastAPI**. It is the part that receives requests from the browser and returns answers.

Main files (all inside `backend/`):

- `app.py` — The front door. Lists all the web addresses (**endpoints**) the browser can call, such as `/api/audit`, `/api/complaints`, `/api/auth/login`.
- `pipeline.py` — Wires the 10 audit agents together.
- `config.py` — All settings and file paths in one place.
- `condition_eval.py` — The rule engine that checks True / False / Unknown.

- `retrieval.py` — The smart search used by the RAG agent.
- `agents/` — One file per audit agent.
- `auth.py` — Login, password security, tokens, and roles.
- `dbt.py` — Reads the beneficiary list and payment ledger.
- `complaints.py` — Stores complaints in a file and enforces who can see what.
- `trace/` — The 7 money-trace agents.

The main endpoints (web addresses)

- Audit group: `/api/audit`, `/api/benchmark/run`, `/api/report`, `/api/leaderboard`.
- Grievance group: `/api/auth/login`, `/api/complaints`, `/api/trace`, `/api/beneficiaries/...`, `/api/dbt/transactions`.

---

7. Part E — The Frontend (what the user sees)

The frontend is plain **HTML + CSS + JavaScript** — no complicated build tools.

- `index.html` + `app.js` + `styles.css` — The **Benchmark Dashboard**. It shows dataset statistics, lets you audit a case, run the benchmark, view ablations, browse schemes, and see the leaderboard. A login screen appears first.
- `grievance.html` + `grievance.js` — The **Grievance & Money-Trace portal**. Citizens file complaints and view the traced result; officers see all complaints and update their status.

Both pages talk to the backend using simple "fetch" calls (asking for JSON data).

---

8. Part F — The folder map

```
YojanaBench/
■■■ backend/          The brain (Python + FastAPI)
■   ■■■ agents/       The 10 audit agents
■   ■■■ trace/        The 7 money-trace agents
■■■ frontend/         The face (HTML/CSS/JS pages)
■■■ data/             The knowledge (schemes, guidelines, cases, QA, DBT ledger)
■■■ scripts/          Helper tools to rebuild the dataset
■■■ tests/            Automatic checks that prove the code works
■■■ docs/             Documentation (including this file)
■■■ run.py            The one command that starts everything
■■■ benchmark.py      Command-line tool to score a model
■■■ requirements.txt  The list of Python libraries to install
■■■ README.md         The technical overview
```

9. How it flows — two examples

Example 1: Auditing an application

1 You open the dashboard and pick a case.

2 The browser sends the case to `/api/audit`.

3 The backend runs the 10-agent pipeline: load rules → clean input → find clauses → compute eligibility & benefit → double-check → final verdict.

1 The dashboard shows the verdict, the amount, the quoted clauses, and the recommended next step.

Example 2: Tracing missing money

1 Citizen logs in and files a complaint ("housing money never came", UTR given).

2 The browser sends it to `/api/complaints`.

3 The backend runs the 7 trace agents: find the payment → check where the money went → compare amounts → detect fraud → produce a verdict and a draft notice.

1 The citizen sees the result; if money was diverted, an officer sees a red alert and a **mule-account warning**, and can move the complaint forward.

10. Glossary (simple words)

- **Benchmark** — A fixed set of test questions with correct answers, used to score the system fairly.
- **Agent** — A small step in the pipeline that does one job.
- **Pipeline** — Agents chained in order, output of one becomes input of the next.
- **Rule engine / condition eval** — Code that checks rules and answers True / False / Unknown.
- **Verdict** — The final decision (eligible / ineligible / needs_info).
- **needs_info** — We cannot decide yet because some information is missing.
- **RAG (retrieval)** — Finding the right rule documents before answering.
- **DBT** — Direct Benefit Transfer; government money sent to a bank account.
- **UTR** — Unique Transaction Reference; like a receipt number for a payment.
- **Sanctioned amount** — The amount the government approved to pay.
- **Credited amount** — The amount that actually reached a bank account.

- **Mule account** — One bank account used to collect money for many different people, a sign of organised fraud.
- **Token** — A temporary digital ID given after login so you stay signed in.
- **Clause** — A small numbered piece of an official rule document.
- **Endpoint / API** — A web address the frontend calls to ask the backend to do something.

---

11. Frequently asked questions

Q: Does it need the internet or paid AI?

No. It runs fully offline. A local AI is optional and only improves wording.

Q: Are the numbers real?

No. The scheme amounts are **simplified, illustrative values** for a reproducible demo. Replace them with official rates for real use.

Q: Where is the data stored?

In plain JSON/text files under `data/`. Complaints and users are stored under `runtime/` (created automatically).

Q: Is it safe?

It is a college demo. Passwords are hashed and logins use tokens, but it is not hardened for production use.

Q: How do I start it?

See the separate **Setup Guide** document (Setup_Guide.md / Setup_Guide.pdf).